Introduction

Bounded Research As Infrastructure

Bounded Research As Infrastructure I

AutoResearch systems are most useful when their planning, evidence, evaluation, and review surfaces remain inspectable. The recent pattern popularized by bounded coding-agent research loops is simple: define a tractable objective, try candidate changes under a budget, keep the result that improves the metric, and leave a replayable trace of what happened (Karpathy 2026). That pattern is powerful, but it is also easy to overstate. A public research template should show how to run the loop without hiding cost, evidence, review, or execution boundaries.

Exemplar Task

This project implements that safer version. The central task is `small MNIST neural-network classification: MNIST handwritten digit database` from the handwritten-digit database (LeCun et al., n.d.), a `nearest-centroid` baseline, and a finite list of candidate model families (`MLP, nearest-centroid, softmax regression, tiny patch-attention`). The AutoResearch loop is responsible for proposing candidate configurations, evaluating them against `test_accuracy`, selecting the best result with deterministic tie-breaking, and writing the evidence needed to review the claim.

Contribution And Boundary

Contribution And Boundary I

The contribution is not a new MNIST classifier. It is a template-level demonstration of how bounded AutoResearch can be orchestrated through the same lifecycle used for reproducible papers: tests run first, analysis writes structured artifacts, rendering hydrates manuscript variables, validation checks evidence and readiness, and copy stages publish final deliverables. The default path makes no network calls, no LLM calls, executes no generated code, and never treats a generated review packet as human approval. The methods in sec. sec:methodology define that boundary, and the results in sec. sec:results report only what the validated artifacts support.

Process Organization Motif

The exemplar also treats research orchestration itself as a first-class research object. In that limited operational sense, "research for research's sake" means that programs, ledgers, evidence registries, validation gates, and manuscript hydration do more than report a result: they maintain the conditions under which the next research step can be inspected, replayed, criticized, and extended. This is a process analogy, not a moral claim that software is an end in itself or a biological claim that the template is alive.

Related Work And Current Trends

Information Overload And Machine-Readable Science I

The current AutoResearch literature is driven by a practical bottleneck: scientific output is growing faster than document-centered review and synthesis can absorb. Recent science-of-science evidence complicates any simple productivity story: AI-augmented scientists can publish and be cited more often, but the same adoption pattern can narrow the collective range of topics and interactions in science (Hao et al. 2026). This is a 2026 Nature result, not a 2024 analysis, and it motivates governance rather than celebration.

One response is to make literature synthesis more source governed. OpenScholar uses retrieval-augmented generation over a large scientific passage store and reports citation accuracy improvements on literature synthesis tasks (Asai et al. 2026). PaperQA2 similarly evaluates literature-search agents against expert scientific tasks and emphasizes cited answers, contradiction detection, and factuality (Skarlinski et al. 2024). STORM, PaperQA, and GPT Researcher are adjacent source-grounded writing systems that motivate this project's insistence on citation-backed claims and visible evidence surfaces (Shao et al. 2024; Lala et al. 2023; Contributors 2026). The common lesson is that automated writing is not enough: claims must remain tied to inspectable sources, artifacts, and evaluation records.

Structured Distillation And Conceptual Knowledge Substrates I

The Discovery Engine proposes a more structural answer to the same overload problem. It distills publications into source-linked knowledge artifacts, organizes those artifacts under a conceptual schema, encodes them into a high-dimensional Conceptual Tensor, and unrolls that tensor into graph and vector views for agent navigation (Baulin et al. 2025). This project does not construct a Conceptual Nexus Model or claim domain-scale literature synthesis. It adopts a much smaller analogue: outputs, ledgers, evidence registries, figure records, and hydrated manuscript variables are file-backed objects whose provenance can be checked before publication.

Structured Distillation And Conceptual Knowledge Substrates I

The representational background is broader than one framework. FAIR principles argue for data that are findable, accessible, interoperable, and reusable by people and machines (Wilkinson et al. 2016). RO-Crate and Workflow Run RO-Crate package research artifacts and computational executions with linked-data metadata (Soiland-Reyes et al. 2022; Leo et al. 2024). Hyperdimensional computing and vector symbolic architectures provide one route for robust high-dimensional symbolic-subsymbolic representations (Heddes et al. 2024), while tensor factorization methods such as TuckER and mixed-geometry tensor factorization show how multi-relational knowledge graphs can be completed and queried as structured tensors (Balazevic et al. 2019; Yusupov et al. 2025). The local contribution here is not a new knowledge representation method; it is an executable template that makes a small research workflow

compatible with that machine-readable direction.

End-To-End AutoResearch Systems I

The most ambitious AutoResearch systems now aim at the entire scientific lifecycle. The AI Scientist assembles idea generation, experiment execution, paper writing, and automated review (C. Lu et al. 2024), and its Nature version reports an end-to-end AI research pipeline whose generated manuscript passed a workshop peer-review round (Lu et al. 2026). AI Scientist-v2 removes more hand-authored scaffolding and uses agentic tree search for broader hypothesis exploration (Yamada et al. 2025). FutureHouse's platform exposes specialized scientific agents for literature search, deep review, novelty checking, and chemistry planning (FutureHouse 2025), while Robin integrates literature and data-analysis agents in a lab-in-the-loop discovery workflow (Ghareeb et al. 2026).

Survey work is already separating reliable assistance from risky autonomy. The AI for Auto-Research roadmap describes the full lifecycle from creation to dissemination, but stresses that novelty, research-level implementation, and judgment remain fragile under automation (Kong et al. 2026). EXHYTE frames discovery as an iterative Exploration, Hypothesis generation, and Testing loop, clarifying where current systems are mature and where closed-loop autonomy remains thin (Hasib et al. 2025). This exemplar therefore takes the opposite stance from full autonomy: it implements a bounded local loop whose candidate space, data, cost, outputs, and review gates can be audited.

Autoformalization supplies a different kind of boundary: instead of only asking whether generated text is plausible, it asks whether an informal statement can be translated into a form that a proof assistant or compiler can check. AlphaProof shows the power of reinforcement learning over formal mathematical proof search (Hubert et al. 2025). Process-driven autoformalization in Lean uses compiler feedback as a precise signal for improving translations from natural-language mathematics to formal statements and proofs (J. Lu et al. 2024). APOLLO turns Lean feedback into an iterative proof-repair workflow in which generated proofs are decomposed, patched, and reverified (Ospanov et al. 2025).

Autoformalization And Verification I

This project does not perform theorem proving, proof repair, or formal mathematical verification. It borrows the architectural lesson: generated research artifacts should be checked by deterministic tools with explicit error surfaces. Here those tools are test suites, schema checks, evidence registries, render validation, source hygiene greps, and review packets rather than Lean, Isabelle, or Coq.

ML For ML And Evolutionary Algorithm Discovery I

ML-for-ML systems optimize models, code, or algorithms with search loops that are themselves subject to evaluation. Karpathy's `autoresearch` repository frames a minimal version of this pattern as a prompt-controlled system with a fixed budget, editable code surface, and comparable metric (Karpathy 2026). MLAgentBench and MLE-bench package machine learning tasks as scored, replayable environments with logs and grading outputs (Huang et al. 2023; Chan et al. 2024).

At a larger scale, AlphaEvolve couples language-model proposals to evolutionary program search and automated evaluators, producing algorithmic improvements in mathematics and computing (Novikov et al. 2025). DeepEvolve adds external retrieval, multi-file code editing, and debugging to the same basic proposal-implementation-evaluation loop (Liu et al. 2025). This manuscript's candidate search is intentionally smaller: a finite list of configured model families is evaluated against `test_accuracy`, with deterministic selection and recorded deferrals rather than unbounded code mutation.

Agentic Science, Graph Retrieval, And Epistemic Foraging I

Agentic science surveys describe systems that move from tool use toward scientific agency across perception, knowledge representation, planning, experimentation, analysis, and communication (Wei et al. 2025; Gridach et al. 2025). GraphRAG work adds structured retrieval to that picture: graph construction and graph-aware retrieval can support multi-hop reasoning, but benchmarks also show that knowledge-graph RAG remains brittle when relevant knowledge is incomplete (Xiao et al. 2025; Zhou et al. 2026). Active-inference perspectives make a similar design demand in different language: scientific agents need persistent uncertainty-aware memory, causal models, counterfactual exploration, deterministic validation, and human judgment as an architectural component (Duraisamy 2025).

This exemplar uses those ideas as constraints, not as capabilities it already possesses. It does not run live literature mining, autonomous proof search, external agent swarms, graph-based hypothesis hunting, or self-approval. Instead it exposes bounded candidates, explicit budgets, local evidence links, local `MNIST` input data, benchmark-style scoring, source-linked figures, manuscript hydration, and human review gates. That is the intended safe baseline for a public template.

Benchmark, Documentation, And Process Analogies I

`MNIST` and LeNet remain useful here because they provide a compact historical benchmark for small neural networks and handwriting recognition (LeCun et al. 1998, n.d.). Vision Transformers introduce the patch-token pattern for image classification at scale (Dosovitskiy et al. 2020); this exemplar borrows only the patching and attention representation through `Tiny patch-attention classifier`, then keeps the implementation inside the configured candidate budget. MLPerf Tiny and OpenML motivate explicit task descriptions, fixed inputs, machine-readable run metadata, and checkable metrics (Banbury et al. 2021; Vanschoren et al. 2014). Machine-learning reproducibility checklists motivate reporting data, seeds, model sizes, hyperparameters, and compute boundaries (Pineau et al. 2020).

Benchmark, Documentation, And Process Analogies I

Dataset and model documentation work further informs the safe boundary. Datasheets for Datasets motivate explicit reporting of dataset motivation, composition, collection, and recommended use (Gebru et al. 2021). Model Cards motivate structured reporting of model context, intended use, evaluation procedure, and limitations (Mitchell et al. 2019). The diagnostic layer follows the same conservative reporting posture: calibration is treated as separate from accuracy (Guo et al. 2017), binomial accuracy intervals use Wilson-style score intervals (Wilson 1927), matched classifier comparison is summarized through paired discordance (Dietterich 1998), deterministic bootstrap intervals are local resampling diagnostics (Efron and Tibshirani 1993), and probability quality is reported with Brier score, negative log likelihood, and chance-corrected agreement (Brier 1950; Cohen 1960).

Benchmark, Documentation, And Process Analogies I

The process language is borrowed cautiously from teleology and theoretical biology. Kant's account of organized beings treats a natural purpose as a whole whose parts and whole mutually condition one another (Ginsborg 2022). Autopoiesis characterizes living systems through self-producing organization (Varela et al. 1974), and later work connects Kantian natural purpose to autopoietic individuality (Weber and Varela 2002). Moreno and Mossio's account of biological autonomy emphasizes organizational closure and self-maintenance (Moreno and Mossio 2015). This paper uses those ideas only as disciplined analogies for configured scientific workflows whose artifacts help reproduce, constrain, and evaluate subsequent artifacts.

Security and supply-chain references enter with the same restraint. NIST's zero-trust architecture treats verification as explicit and continuous rather than inherited from a trusted perimeter (Rose et al. 2020). The NIST Secure Software Development Framework emphasizes repeatable practices for reducing software vulnerability risk (Souppaya et al. 2022). SLSA frames software-artifact provenance and supply-chain integrity as a graded assurance problem (Open Source Security Foundation 2026), while MITRE ATT&CK T1195 names supply-chain compromise as a concrete adversary technique (MITRE ATT&CK 2026). This exemplar borrows those frameworks as disciplined analogies for local research artifact integrity: checksums, inventories, review gates, and explicit non-claims, not production deployment certification.